GreensKeeper: An Automated Plant Monitoring and Care System

Senior Design Report



Nyra Graham
Advisor: Prof. Douglas MacMartin

May 17, 2025

1. Introduction

GreensKeeper is an automated plant care and monitoring system designed to simplify the challenges of houseplant maintenance. Many plant owners struggle with inconsistent watering, poor lighting conditions, or lack of real-time information about their plant's needs. GreensKeeper addresses these issues by integrating hardware and software into a compact, self-contained shelf system. Each slot on the shelf supports a live plant and includes a soil moisture sensor, controllable grow light, and water pump. All components are managed through a Raspberry Pi and presented via a Flask-based web interface that allows for manual and automatic control. This project was inspired by the increasing demand for smart home automation systems and the growing interest in sustainable indoor gardening. By combining IoT principles with simple horticultural requirements, GreensKeeper empowers users to remotely monitor and support plant health with minimal daily involvement.

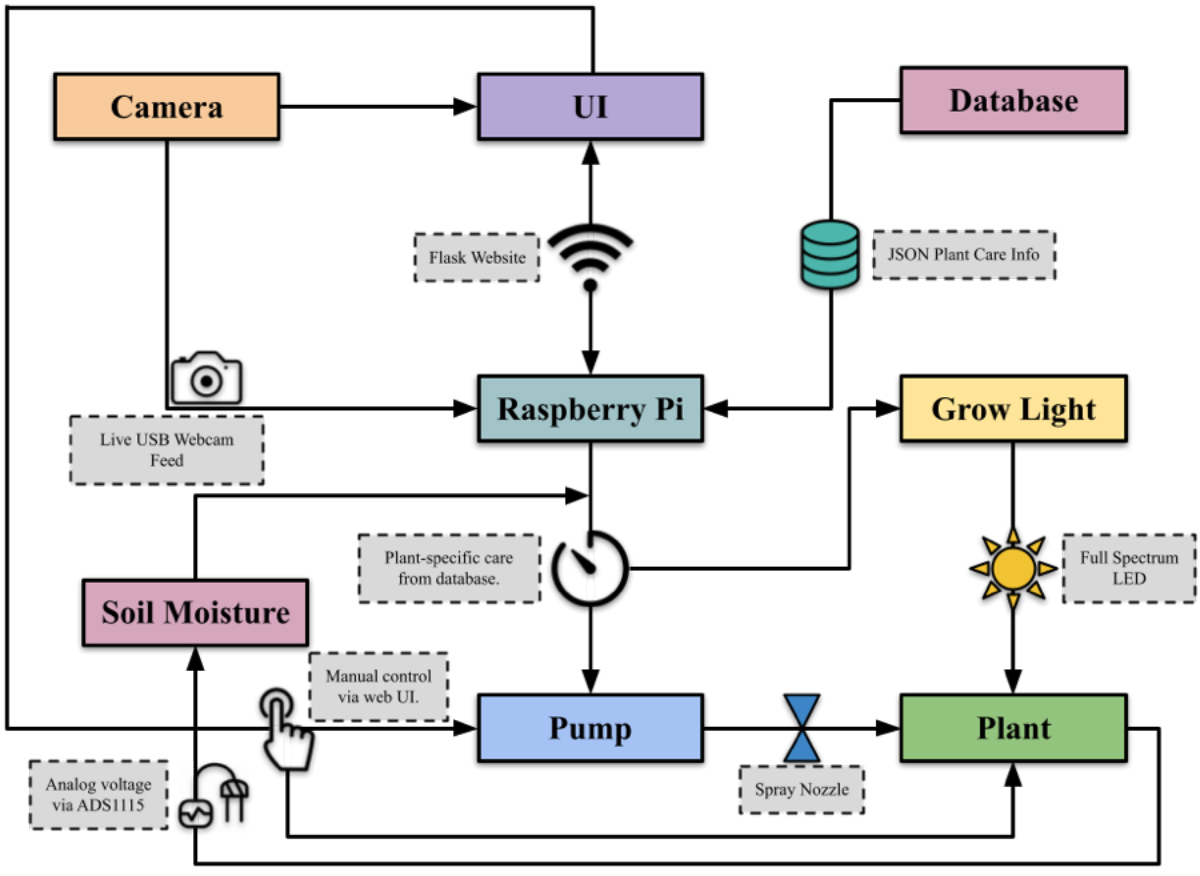


Figure 1: System-level diagram of GreensKeeper showing data flow between sensors, actuators, and interface components.

2. System Overview

The GreensKeeper system integrates a variety of sensors, actuators, and software components to provide automated plant care through a self-contained smart shelf. At its core is a Raspberry Pi, which coordinates data acquisition, decision logic, and user interface operations.



Figure 2: Photo of the physical GreensKeeper setup with grow light, sensor, and watering system in place.

Each plant slot on the shelf is equipped with a soil moisture sensor connected via an ADS1115 analog-to-digital converter to the Raspberry Pi. This allows for precise analog readings, enabling plant-specific watering decisions based on real-time humidity levels. A 12V peristaltic pump, driven through a motor controller and GPIO control, delivers water to the plant when dry soil is detected. The moisture threshold and watering frequency are adjustable per plant.

Lighting is controlled by a PCA9685 PWM driver connected to the Raspberry Pi via the I extsuperscript2C bus. Each grow light's brightness level (off, low, medium, high) is

specified in a local plant database and scheduled according to the plant's required sunlight hours. Power to the pumps and lights is supplied by a 12V source, while the Raspberry Pi and logic components use 3.3V or 5V as appropriate.

All data and controls are presented to the user through a custom-built Flask website hosted on the Raspberry Pi. This web interface includes real-time plant data, a live camera feed, and manual overrides for light and watering systems. A JSON database maintains plant-specific care instructions, which the automation logic uses to schedule lighting and irrigation.

Subsystem	**Component**	**Interface**	**Function**
Sensing	Capacitive Soil Moisture Sensor	Analog via ADS1115	Detects current soil humidity and sends data to the Pi
Control	Raspberry Pi 4	–	Runs automation logic and hosts the Flask server
Lighting	Full-Spectrum LED Panel	PWM via PCA9685	Provides scheduled plant-specific lighting levels
Irrigation	12V Peristaltic Pump	GPIO via motor driver	Waters the plant when dryness is detected or on schedule
Interface	Flask Web Interface	Local Wi-Fi / Browser	Displays live camera feed and sensor data, allows manual override
Database	JSON File	File read/write	Stores per-plant lighting and watering needs

Table 1: Subsystems and components with interface and function.

The system is modular and scalable: although the current implementation supports one plant, the architecture allows for expansion to additional slots with minimal code and wiring changes.

3. Design Requirements and Goals

The GreensKeeper system was developed to meet clear functional and performance goals while operating within a set of technical and practical constraints.

Functional Goals

- Automate plant watering based on real-time soil humidity readings.
- Deliver adjustable light intensity tailored to each plant's needs.

- Provide a live video feed and control panel via a local web interface.
- Store plant care parameters in a readable and editable JSON database.
- Operate independently with scheduled control actions (e.g., daily lighting).
- Respond to user commands with minimal latency (¡2 seconds).
- Allow for future expansion to more plant slots with modular design.

Constraints and Design Considerations

- The system must be powered by a single 12V supply, with logic-level conversion.
- All components must be low-cost, readily available, and compact.
- GPIO pin count is limited, so I^2C peripherals (ADS1115, PCA9685) were prioritized.
- The user interface must be intuitive and accessible via a web browser.
- Sensors must be mounted directly in soil and remain reliable over time.
- The system should be desktop-sized and visually unobtrusive.

Design Justifications

To meet these goals and constraints:

- A Flask web server was chosen for its lightweight integration with Python and Raspberry Pi GPIO control.
- JSON was used as the plant database format due to its readability and flexibility.
- PWM-based brightness control was achieved via the PCA9685, which offers multiple channels and I^2C communication.
- The soil moisture sensor was connected via an ADS1115 to enable high-resolution analog input on a Pi.
- Components were prototyped on a breadboard and mounted to a wood shelf with custom 3D-printed brackets.

4. Hardware Design

The hardware design of the GreensKeeper system integrates sensing, actuation, and control components on a compact shelf structure. This section details the electronic and mechanical elements involved in monitoring and maintaining plant health.

Mechanical Design

The frame consists of two 6×1×24" pine boards joined by custom 3D-printed legs and brackets. These parts were printed using matte black PLA and provide support for wiring, lights, and tubing. The top shelf holds all electronics and provides mounting for the grow light, while the bottom shelf supports the plant pots. Figure 3 shows the rendered CAD model of the shelf, and Figure 4 shows the 3D-printable layout used to fabricate the legs.



Figure 3: CAD model of the physical shelf used to house the GreensKeeper system.

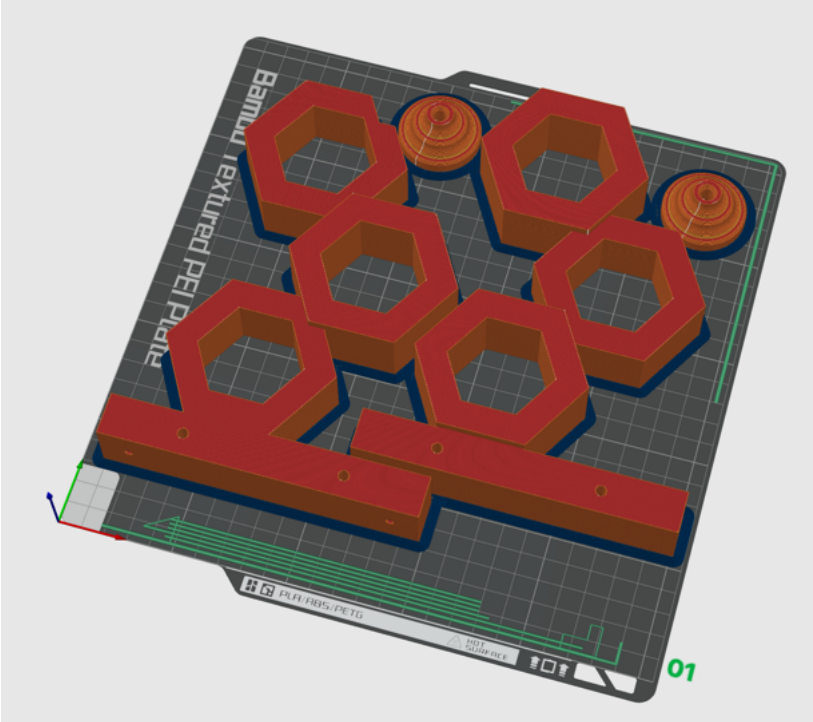


Figure 4: 3D-printable components arranged on build plate. Parts include hexagonal support segments and end brackets.

Electronics and Circuit Design

The electronics are mounted to the top shelf on a prototyping breadboard and powered by a 12V supply. Core components include:

- **Raspberry Pi 4**: central controller, runs Flask server and care logic
- **ADS1115**: analog-to-digital converter for moisture sensor
- **PCA9685**: I^2C PWM driver for brightness control
- **MOSFET (IRLZ44N)**: switches 12V current to LED panel
- **Motor Driver (TB6612FNG)**: controls the 12V peristaltic pump
- **Capacitive Moisture Sensor**: provides analog voltage correlated with soil hydration
- **USB Camera**: streams live feed via web interface

The Fritzing diagram in Figure 5 shows the full wiring setup. Power lines are separated for 3.3V logic and 12V actuation. The ADC and PWM driver are both connected to the I^2C bus, conserving GPIO pins.

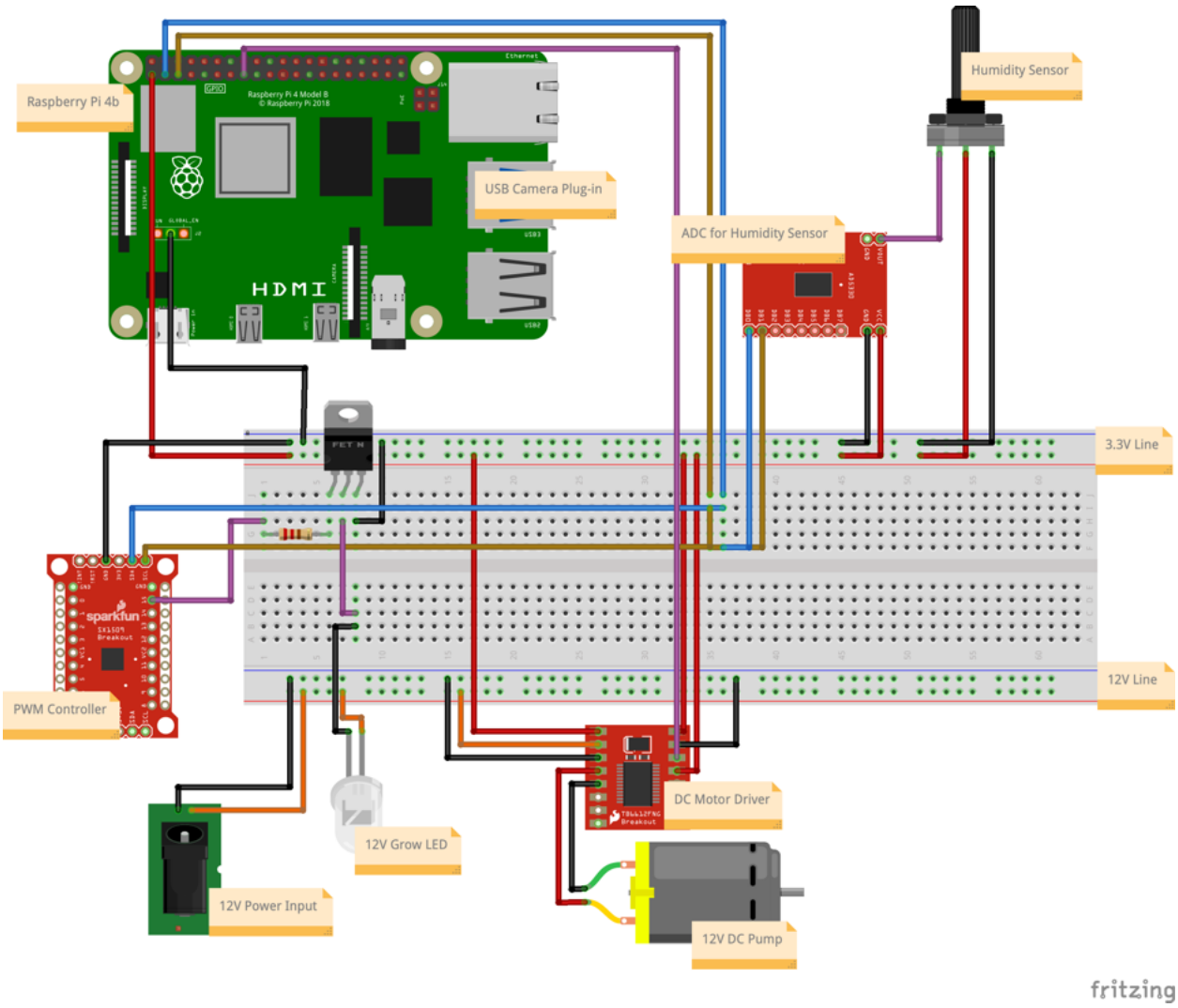


Figure 5: Fritzing diagram of the complete circuit used in GreensKeeper.

5. Software Design

The GreensKeeper software architecture is built around a modular Python codebase hosted on a Raspberry Pi. The software includes control logic for watering and lighting, a local web interface, and a scheduling system for autonomous behavior.

Control Logic

The Raspberry Pi reads humidity values through the ADS1115 using the Adafruit CircuitPython library. Humidity is mapped from voltage to percentage and compared against plant-specific watering thresholds. If the humidity is below threshold, the Pi activates the pump via a motor driver for a short timed interval.

Light brightness is controlled using the PCA9685 PWM driver. Each plant's light level (off, low, medium, high) and sunlight duration (in hours per day) are stored in a local JSON database. The Pi sets the appropriate PWM duty cycle on system boot and schedules lights to turn on each day at 9 AM using APScheduler.

Web Interface

A Flask-based web server runs on the Pi and serves a local HTML interface. The site displays:

- Real-time humidity readings
- Live camera feed (via OpenCV)
- Pump and light status
- Buttons for manual pump and light control
- A per-slot plant assignment interface

Each plant has a dedicated page displaying its care information and current sensor readings. The interface is responsive and minimalistic, styled using Times New Roman with a natural, earthy color palette.

Database and Scheduling

A local JSON database stores all plant care requirements, including watering interval, sunlight hours, and brightness level. These parameters are read at runtime to update GPIO behavior. APScheduler is used to schedule daily lighting control and periodic care checks.

A separate background process script (care_scheduler.py) runs alongside the Flask server. It handles all timed automation such as checking when to water and turning off lights after their required duration.

System Integration

All scripts are written in Python and communicate via shared files (e.g., light_status.txt, last_watered.json). This enables asynchronous but synchronized behavior between the real-time control and the user-facing interface. The system is designed to boot and begin functioning automatically on startup.

6. Testing and Validation

GreensKeeper was tested at both the component and system levels to ensure each feature functioned reliably under expected conditions. Testing was conducted in incremental stages throughout the build process.

Sensor and Actuator Testing

Each core component was tested independently prior to integration:

- The capacitive soil moisture sensor was tested in dry and wet soil to observe voltage response. Readings from the ADS1115 were compared against expected values, and linear mapping to percentage was validated.
- The 12V grow light was tested using direct PWM outputs from the PCA9685. Duty cycles corresponding to "low," "medium," and "high" were confirmed visually and measured for accuracy.
- The pump was activated manually via GPIO22, and water delivery was timed and visually confirmed. A 2-second activation period was chosen based on consistent performance.

Website Interface Testing

The Flask-based web interface was tested across multiple devices for responsiveness. Tests confirmed:

- Sensor values displayed correctly and updated in real time.
- The pump and light status updated on the homepage when activated.
- Manual control buttons triggered the appropriate hardware actions.
- The camera stream loaded consistently and reflected live footage.

Automation and Scheduling Testing

APScheduler tasks were tested using both real-time simulation and forced scheduling:

- Light-on events were tested by simulating 9:00 AM conditions and verifying that the appropriate PWM signal was sent and the runtime was logged.

- Automated watering was tested by adjusting the database to reduce the watering interval, confirming the system activated the pump when the threshold was reached.
- Runtime tracking was validated by comparing light on-time to the stored time values, and adjusting database parameters to test cutoffs.

System Integration Testing

Once all modules were integrated, full system tests were conducted with a live plant:

- The soil sensor triggered pump activation under dry conditions.
- The light activated at scheduled times and deactivated after the specified duration.
- The plant care page showed updated values for humidity, light runtime, and last watered timestamp.

Debug messages, log files, and real-time visual verification were used to ensure consistent behavior throughout testing. Overall, the system demonstrated reliable automation and user control in a controlled indoor environment.

7. Results and Reflection

The final GreensKeeper system successfully met its core goals of monitoring and automating plant care for a single shelf slot. The system is fully functional, with real-time humidity readings, automated watering, scheduled lighting, and a live web interface that allows for both manual and automatic control. The modular design also allows for future scalability to multiple plant slots with minimal changes to the codebase and wiring.

The pump and light systems worked reliably in practice. Manual overrides via the website consistently activated the intended hardware, and automated behaviors triggered based on the database schedule. One unexpected success was the runtime tracker, which provided a clear way to visualize the system's lighting history per plant. The plant-specific JSON database proved to be a lightweight and flexible way to encapsulate care logic, especially when compared to cloud-based solutions.

That said, a few limitations and lessons emerged during development. Initially, integrating the live USB camera stream into the Flask application was challenging, particularly in balancing resource use with responsiveness. Wiring also became difficult to manage as components were added; this was addressed late in the project by reorganizing the breadboard and shifting to labeled power rails.

The humidity sensor chosen for testing occasionally produced noisy readings. Although the analog-to-digital converter improved precision, sensor calibration and consistency could be improved in a future version. Additionally, the system was tested primarily in controlled indoor conditions; exposure to outdoor variability or long-term plant growth would require further testing and robustness improvements.

This project ultimately deepened our understanding of embedded systems integration, real-time I/O control, and practical web-based user interfaces. It provided a unique opportunity to bridge digital control with biological systems in a way that is both technically rigorous and approachable for non-technical users.

8. Future Work

While the current GreensKeeper system demonstrates core functionality with one plant slot, several enhancements and expansions are planned to improve usability, scalability, and performance.

- **Multi-slot expansion**: The system was architected to support multiple plants. Future work includes wiring additional sensors, pumps, and lights, and assigning each to its own slot in the database and UI. This would allow for fully independent care routines per plant.
- **Sensor calibration and redundancy**: Improving soil humidity sensor accuracy and adding redundancy (e.g., multiple sensors per pot or soil temperature) could increase watering precision. Incorporating external ambient light sensors could also allow the system to adapt to environmental conditions.
- **Event logging and data visualization**: Currently, runtime and watering data are stored as text. Future versions could store event logs and sensor history in a database, then visualize them in the UI with charts or graphs to track plant health over time.
- **Mobile and remote access**: Hosting the Flask server behind a secure public address or integrating with a cloud dashboard would allow for off-network access via phone or laptop. A lightweight mobile app or shortcut page could further improve ease of use.
- **Automated plant identification**: In the long term, integrating a camera with image recognition could allow the system to detect what plant is in a slot and load default care parameters automatically.
- **Physical enclosure and modular kits**: Replacing the breadboard prototype with a custom PCB and designing a modular case for each plant unit would allow the system to be deployed as a standalone device for home use or in educational settings.

These additions would help make GreensKeeper more robust, adaptable to different environments, and accessible to a broader audience.

9. Appendices

- Code can be seen here: https://github.com/nyragraham/GreensKeeper

Component	**Description / Function**	**Qty**	**Estimated Price**
Raspberry Pi 4	Main controller	1	**$55.00**
MicroSD Card (16GB+)	OS and storage	1	**$10.00**
USB Webcam	Live camera feed	1	**$25.00**
PCA9685	PWM driver for light	1	**$9.00**
ADS1115	Analog to digital converter	1	**$8.00**
Capacitive Moisture Sensor	Humidity reading	1	**$4.00**
IRLZ44N MOSFET	Light switching	1	**$2.00**
Peristaltic Pump (12V)	Delivers water	1	**$12.00**
TB6612FNG Motor Driver	Pump control	1	**$7.00**
LED Grow Light (12V)	Plant lighting	1	**$10.00**
12V Power Supply	Powers pump/light	1	**$12.00**
Jumper Wires + Breadboard	Prototyping	–	**$8.00**
Wood Boards (6×1×24 in.)	Shelf structure	2	**$8.00**
3D Printed Brackets	Frame support	Varies	**$2.00**
Tubing + Spray Nozzle	Directs water to plant	1 set	**$5.00**

Table 2: Estimated Bill of Materials for the GreensKeeper system. Bolded prices are estimated.